

Tier Slider, Browser Back Navigation & Code Quality Pass

Business Requirements Document · v1.0 · 1 October 2026

Item	Detail
Owner	Sukrit Srivastava (build lead)
Reviewers	Ayushmaan (Operations Head, UAT) · Rahul (Founder, approval)
Implemented by	Claude Code, in the existing PrintEvr repo
Status	Ready to build. Localhost demo first; nothing is pushed or deployed until Sukrit approves.
Builds on	Pricing Calculator BRD v2.1 (FR-2 tier matching, FR-3 next-tier nudge) and the Cart & Invoice BRD v1.0. Both are already implemented; this BRD covers only the three changes below.

Summary

This release makes three changes to the live calculator:

#	Change	Outcome
A	Replace the volume tier bar with an interactive tier slider	Every price break is marked with its unit price; the next break and the "overpay zone" are highlighted, and the prompt to reach the next break updates instantly while dragging.
B	Make the browser Back and Forward buttons work	Each screen gets its own URL. Back returns to the last open screen with its state intact instead of leaving the site.
C	Full code review and optimisation pass	Claude Code reviews the entire codebase and makes it faster and cleaner without changing any price, total, GST or invoice output.

1. Background and problem

Tier bar. The calculator currently shows a static tier ladder under the quantity input. It marks the breakpoints but does not show what each break is worth, and users have to type quantities to discover savings. Printevr's catalogue has steep drops between tiers. For a Rigid Box 3 × 3 × 2 in Top-Bottom, 250–499 boxes cost ₹75 each and 500+ cost ₹55 each, so 450 boxes (₹33,750) cost ₹6,250 more than 500 boxes (₹27,500). Staff should see that at a glance while quoting.

Back button. The app is a single-page React app that swaps screens using component state. The URL never changes, so the browser has no history inside the app. Pressing Back (or the Android back gesture, or the iOS swipe) leaves the website entirely and loses the user's work.

Code quality. The calculator, cart, invoice and GST features were built quickly across several sessions. Before Printevr uses the tool full-time, the whole codebase needs one consolidated review for efficiency, duplication, dead code and robustness.

2. Objectives and success measures

Objective	Measure of success
Make price breaks obvious	On any standard item, every tier breakpoint and its unit price is visible on the slider without scrolling or hovering.
Prompt towards savings instantly	The next-break prompt updates on every drag frame (within 16 ms), with no network wait.
Stop overpaying quotes	Whenever a higher tier is cheaper in total, the slider shows the overpay zone and the "Use N" banner (matches existing FR-3 better_option).
Working browser navigation	Back and Forward move between screens inside the app; refresh or a pasted link opens the same screen and configuration.
No regressions	100% of existing tests (pricing golden tests, cart, invoice, GST selector) pass before and after.
Faster, cleaner code	Measured before/after: bundle size, API p95 latency, typecheck and lint errors. None may get worse; the review report lists every change.

3. Scope

In scope

- Tier slider on the calculator screen, for standard and custom sizes, including all edge states in FR-A9.
- One additive API change: a tier_schedule block in the /api/calculate response (Section 7).
- Client-side routing for every existing screen, URL-held calculator state, SPA fallback for deep links on Vercel.
- Whole-codebase review and behaviour-preserving optimisation, with a written report.

Out of scope

- Any change to pricing rules, tier matching, custom-size interpolation, GST logic, invoice layout or invoice numbering.
- New screens, new features, login, or visual redesign beyond the slider.
- Changes to the price sheet in data/ or to config/products.yaml values.
- Dependency major-version upgrades (these may be proposed in the review report, not applied).

4. Requirement A: Tier slider with price-break highlighting

The slider replaces the existing tier ladder component (named TierLadder in BRD v2.1). The numeric quantity input and its +/- buttons stay, directly above the slider, and the two stay in sync both ways.

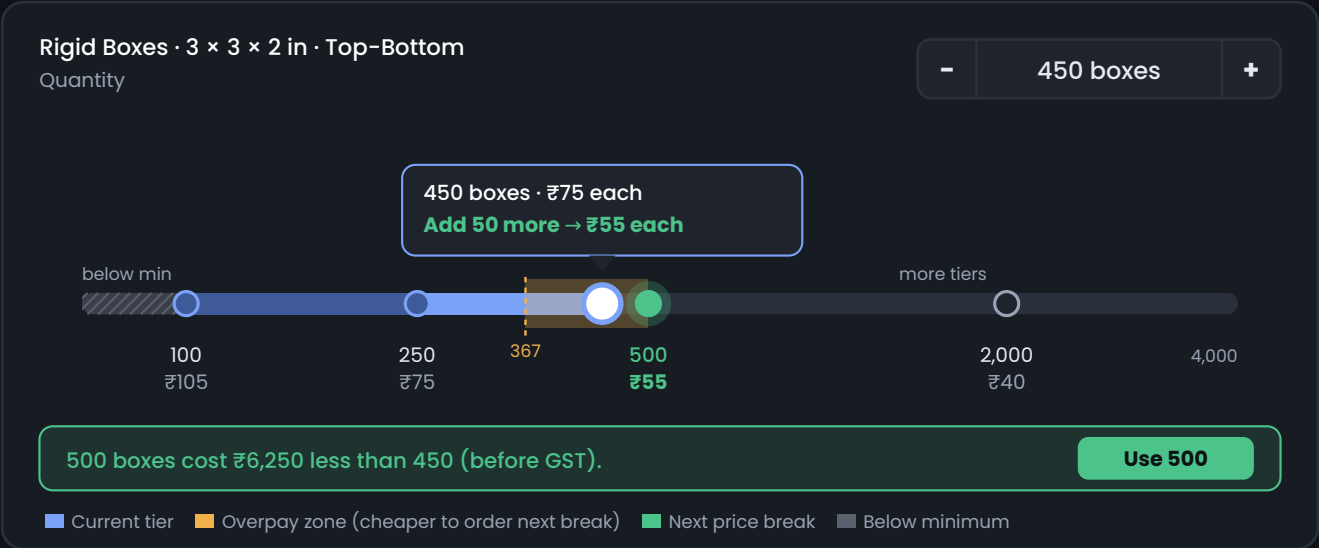


Figure 1. Target design, Rigid Box 3 × 3 × 2 in Top-Bottom at 450 boxes. Prices shown are from the 2025–26 sheet; the "more tiers" segment stands for whatever breakpoints the sheet has between 500 and 2,000. Colours are indicative; match the app's existing theme.

4.1 Functional requirements

FR-A1	Placement and sync. Slider sits under the quantity input on the calculator. Dragging updates the input; typing or +/- moves the thumb. Typed values beyond the slider's range pin the thumb at the end and label it (e.g. "5,000 · beyond scale").
FR-A2	Scale. Use a tier-segmented scale, not a linear one: each tier gets an equal-width segment, and quantity is linear within a segment. This keeps small tiers (100, 250) as usable as large ones (2,000). If the product bills below minimum (<code>bill_at_min</code>), a narrower "below minimum" segment from 1 to the first tier sits at the left. The right end is <code>max_qty</code> when the product has one, else <code>top tier × slider_max_multiplier</code> (default 2, in config).
FR-A3	Price-break markers. Every tier breakpoint gets a marker labelled with its quantity and unit price ("500 · ₹55"). Breakpoints already passed use the progress colour; the next breakpoint is highlighted (distinct colour plus a soft glow ring); later ones are neutral. Tapping or clicking a marker sets the quantity to exactly that breakpoint.
FR-A4	Current tier. The segment of the tier currently applied is filled with the accent colour up to the thumb; the range is shown in the receipt as today ("250 qty, applies to 250–499").
FR-A5	Immediate prompt. A chip above the thumb shows " <i>qty</i> · ₹ <i>unit</i> each" and, when a higher tier exists, "Add <i>n</i> more → ₹ <i>next</i> each". It updates on every pointer-move frame, computed in the browser from <code>tier_schedule</code> (no API wait). The chip keeps the existing FR-3 wording so the chip and receipt never disagree.
FR-A6	Overpay zone. Inside each tier, shade the quantities where ordering a higher breakpoint costs less in total (from <code>overpay_from</code> to the end of that tier). While the thumb is inside the zone, the next marker pulses and the existing better-option banner shows with its "Use N" button. The button changes quantity only when clicked (FR-3 rule 3 still holds).
FR-A7	Snapping. On drag release, if the thumb is within 1.5% of track width of a breakpoint, snap to the breakpoint. Keyboard and typed input never snap.
FR-A8	Keyboard and screen readers. Arrow keys ±1 unit; Page Up / Page Down jump to the next / previous breakpoint; Home / End go to minimum / slider end. The control exposes <code>role=slider</code> with <code>aria-valuetext</code> such as "450 boxes, ₹75 each, 50 short of the ₹55 tier". Use a native input <code>type=range</code> underneath (mapping position to quantity) or an equivalent fully accessible custom control. No slider library.

FR-A9	Edge states. (a) Below minimum: thumb in the hatched segment, existing MOQ_APPLIED warning, chip says "Billed as 100". (b) Top tier: no "Add n more" line. (c) max_qty products (sticker sheets, paper printing): slider ends at max_qty; a typed value above it shows the existing manual-quote state. (d) Outdoor branding: quantity comes from $W \times H \times \text{pieces}$, so the slider is read-only (markers and position only) and no overpay zone (suggest_more: false). (e) Custom sizes: markers show the interpolated per-tier prices from the server. (f) Single-tier items: slider without markers or chip. (g) Flagged prices: existing warning banner, unchanged.
FR-A10	Network behaviour. The slider never waits for the network. Totals still come only from /api/calculate, called 250 ms after the last change with the previous request cancelled (existing rule); the receipt dims while loading. Dragging from 100 to 2,000 must cause at most one request per 250 ms pause.
FR-A11	Source of truth. No price or breakpoint is hard-coded in the frontend. The browser computes only display values (current tier, units to next, whether the thumb is in an overpay zone). If the server response disagrees with the chip, the server wins and a dev-mode console warning is logged.
FR-A12	Mobile. Thumb touch target at least 44×44 px. On narrow screens (under 420 px) marker labels collapse to price only, and the quantity label moves to the tooltip on tap; labels must never overlap. Respect prefers-reduced-motion (no pulse).

5. Requirement B: Browser Back and Forward navigation

Add client-side routing with **React Router** (the one new dependency this BRD approves). Every screen gets a URL; moving between screens adds a browser history entry; editing inside a screen does not.

5.1 Route map

Target mapping. In M0, Claude Code inventories the screens and overlays that actually exist and adjusts the paths to match; the rules below apply whatever the final list is.

Screen (as built today)	Path	State restored on Back / refresh
Pricing calculator	/	Product, size or custom dimensions, options, quantity, add-ons, billing type, all from the URL query
Cart	/cart	Cart lines from the existing cart store (unchanged persistence)
Checkout / invoice details (GST selector, Print Unpaid / Print Paid)	/checkout	Form fields held in state that survives route changes; GST choice kept
Any other full-screen view found in M0	/ <name>	Its own inputs
Unknown path	*	"Page not found" with a link to the calculator

Calculator URL example:

```
/?product=rigid_boxes&item=rigid_boxes/3x3x2-in/top-bottom&qty=450&billing=gst
/?product=rigid_boxes&opt1=top-bottom&custom=3.5x3.5x2in&qty=300
```

5.2 Functional requirements

FR-B1	Screen changes push history. Moving from one screen to another (calculator, cart, checkout and any other found in M0) creates a history entry. Browser Back returns to the previous screen; Forward re-opens the next.
FR-B2	In-screen edits replace, never push. Changing product, options, quantity or dragging the slider updates the URL with replace, debounced to 300 ms. After 20 slider moves, one Back press still leaves the calculator.
FR-B3	State restore. Returning to the calculator restores the exact configuration from its URL. Cart contents and checkout inputs live in a store above the router so they survive route changes. Scroll position is restored on Back and reset to top on new navigation.

FR-B4	Deep links and refresh. Refreshing any route, or pasting a calculator URL into a new tab, opens that screen with that configuration. Invalid or stale query values (e.g. an item that no longer exists) fall back to the calculator's default state with a small notice, never a crash.
FR-B5	Vercel SPA fallback. Add or update <code>vercel.json</code> so every non-API, non-asset path serves <code>index.html</code> . It must not shadow <code>/api/*</code> or static assets. Claude Code inspects the current deployment config and proposes the exact rewrite in its MI report.
FR-B6	In-app back buttons. Any existing "Back" / "Continue shopping" buttons use the router: go back in history when the previous entry is inside the app, otherwise navigate to the parent screen.
FR-B7	Overlays. Full-screen overlays that users perceive as pages (e.g. an invoice preview, if one exists) get their own history entry, so Back closes them. Small dialogs (confirmations, tooltips, dropdowns) do not.
FR-B8	No side effects from navigation. Back, Forward and refresh never generate, re-generate or download an invoice, never allocate a new invoice number, never submit a form, and never clear or duplicate cart lines. Invoice actions happen only on an explicit button click.
FR-B9	Page titles. Each route sets <code>document.title</code> (e.g. "Cart · Printevr") so history menus are readable.
FR-B10	Mobile gestures. Android system back and iOS swipe-back behave exactly like the browser Back button (they use the same history).

Why FR-B5 matters

A plain `npm run dev server` always serves `index.html`, so deep links can look fine locally and still 404 on Vercel. Test deep links against the production build locally (`npm run build` then `npm run preview`) and re-check on a Vercel preview only after Sukrit approves pushing.

6. Requirement C: Whole-codebase review and optimisation

After A and B are working, Claude Code reads the **entire** repository (backend, frontend, config, tests, build and deploy files) and makes it more efficient and maintainable. The rule is **behaviour-preserving**: outputs do not change, only how the code produces them.

FR-C1	Baseline first (in M0). Before any change, record: test counts and results, frontend production bundle size (gzip), <code>/api/calculate</code> p95 latency over 200 requests, typecheck errors, lint errors, console errors on each screen. Save to <code>docs/code-review-baseline.md</code> .
FR-C2	Frozen outputs. No change to any price, total, GST amount, rounding, API field name or meaning (except the added <code>tier_schedule</code>), invoice PDF layout, invoice numbering, or the GST selector options.
FR-C3	Backend checklist. Price sheet and config loaded once, not per request; <code>Decimal</code> used for all money with no float conversions; pricing functions pure and free of I/O; repeated lookups replaced with indexed dicts; input validation complete; consistent error codes; dead code and unused imports removed; duplicated logic merged; business numbers only in config.
FR-C4	Frontend checklist. Remove unnecessary re-renders (measure with React Profiler before memoising); clean up effects and in-flight requests; cache <code>/api/catalog</code> with TanStack Query; lazy-load heavy screens (invoice/PDF code) so the calculator loads first; one money formatter (<code>lib/format.ts</code>) used everywhere; no any in TypeScript; every input labelled; remove unused components, styles and dependencies.
FR-C5	Repo checklist. README run steps accurate for a fresh clone; <code>.env.example</code> lists every variable; no secrets in the repo or in the frontend bundle; CORS restricted to the frontend origin; admin token only from the environment.
FR-C6	Fix vs ask. Fix directly: dead code, duplication, typing, measured performance wins, missing tests. Ask Sukrit first: anything that restructures folders or modules widely, changes a dependency version, adds a library, or changes a data format.
FR-C7	Tests. Every bug found gets a test that fails before the fix and passes after. All existing tests keep passing.
FR-C8	Report. Write <code>docs/code-review-report.md</code> : each finding with file, severity (bug / performance / maintainability / security), what was done (fixed, deferred, needs decision), and the before/after metrics from FR-C1.

7. API change: tier_schedule

`/api/calculate` gains one additive block. Every existing field keeps its name and meaning. The server computes every value, using the same Decimal rules as FR-3 `better_option`, so the slider and the receipt can never disagree.

```
"tier_schedule": {
  "sale_unit": "box",
  "min_qty": 100,
  "below_min_policy": "bill_at_min",
  "max_qty": null,
  "slider_max": 4000,
  "suggest_more": true,
  "tiers": [
    { "qty_from": 100, "qty_to": ..., "unit_price": "105.00", "overpay_from": ... },
    { "qty_from": 250, "qty_to": 499, "unit_price": "75.00", "overpay_from": 367 },
    { "qty_from": 500, "qty_to": ..., "unit_price": "55.00", "overpay_from": ... },
    ...
    { "qty_from": 2000, "qty_to": null, "unit_price": "40.00", "overpay_from": null }
  ]
} // abbreviated: "..." values come from the sheet
```

Field	Rule
unit_price	The tier's unit price including per-unit add-ons currently selected. For custom sizes, the interpolated and rounded price for that tier (FR-4).
qty_to	Next tier's qty_from - 1; null for the top tier (or max_qty when set).
overpay_from	Smallest quantity in this tier whose total exceeds the cheapest total of ordering exactly any higher breakpoint (including add-ons). null when no such quantity exists or suggest_more is false.
slider_max	max_qty if set, else top tier qty_from × slider_max_multiplier (config, default 2).

Worked check. Rigid 3 × 3 × 2 Top-Bottom, 250 tier at ₹75: the cheapest higher option is 500 × ₹55 = ₹27,500. 366 × ₹75 = ₹27,450 (cheaper, not overpaying); 367 × ₹75 = ₹27,525 (overpaying). So `overpay_from` = 367, which agrees with the existing T5 test (450 boxes, better option 500, saving ₹6,250).

```
best_higher = min over higher tiers j of (qty_from_j * unit_price_j + per_order_addons)
overpay_from = floor((best_higher - per_order_addons) / unit_price) + 1
overpay_from = null if overpay_from > qty_to or suggest_more is false
```

This sends the selected item's tier prices to the browser, which BRD v2.1 avoided. The tool is internal and these prices are already in the catalogue Printevr shares with clients; the custom-size anchors, surcharge and flag list stay on the server. See decision D1.

8. Non-functional requirements

Area	Requirement
Slider responsiveness	Thumb and chip update every animation frame while dragging (no visible lag on a mid-range Android phone).
API	<code>/api/calculate</code> p95 stays under 50 ms with <code>tier_schedule</code> added (existing target).
Navigation	Route change renders in under 100 ms; no full page reload between screens.
Bundle size	Net gzip growth from this release at most 25 KB (React Router plus slider). The review pass should offset it where possible.

Area	Requirement
Accessibility	WCAG 2.1 AA: keyboard-operable slider, visible focus, text contrast at least 4.5:1, marker meaning not conveyed by colour alone (labels always present).
Browsers	Current Chrome, Edge, Android Chrome and iOS Safari.
Money	Decimal on the server; the frontend never computes totals, only display hints from <code>tier_schedule</code> .

9. Working rules for Claude Code

Hard rules

1. Work on a new local branch `feature/tier-slider-back-nav` created from the current main branch.
2. **Never** run `git push`, merge into main, or run any deploy command (`vercel`, `vercel --prod`). Vercel builds a deployment for pushes to any branch, so nothing leaves the machine until Sukrit says so.
3. Local commits are allowed, one per milestone, so each milestone can be reviewed or rolled back.
4. Do not edit `.env` files, secrets, or anything in `data/`.
5. The only new dependency approved is React Router (`react-router-dom`, latest version compatible with the project's React). Ask before adding anything else.
6. Run the full test suite at the end of every milestone and do not start the next one until it passes.
7. Stop and ask only when something conflicts with this BRD. Otherwise continue through M5 and stop there.

10. Build plan

#	Milestone	Work	Done when
M0	Survey and baseline	Create branch. Read the whole repo. List every screen, overlay and existing back-type button. Record the FR-C1 baseline.	Baseline file written; all existing tests pass; screen inventory posted in the chat
M1	Back navigation	React Router, routes, URL-held calculator state (replace), store above the router, titles, not-found view, <code>vercel.json</code> rewrite proposal.	N1-N10 pass (Section 11); existing tests pass
M2	<code>tier_schedule</code> API	Compute and return <code>tier_schedule</code> ; unit tests for <code>overpay_from</code> , custom sizes, add-ons, <code>max_qty</code> , outdoor.	A1-A6 pass; response otherwise unchanged (snapshot test)
M3	Tier slider	Slider component replacing the tier ladder, chip, markers, overpay zone, snapping, keyboard, mobile layout; component tests.	S1-S14 pass; old ladder component removed
M4	Review and optimise	FR-C2 to FR-C8 across the whole repo.	R1-R7 pass; report written
M5	Regression and demo	Full test run, production build, both dev servers started.	Localhost URLs, UAT checklist and summary posted; stop and wait

11. Acceptance tests

Figures use the 2025-26 sheet; amounts are before GST unless stated. Automate as unit, component or end-to-end tests where the stack allows; the rest go into the UAT checklist for Ayushmaan.

API (M2)

ID	Input	Expected
A1	Rigid 3×3×2 Top-Bottom, qty 350	tier_schedule lists every breakpoint of the item in order; tier 250 has unit_price 75.00, qty_to 499, overpay_from 367; tier 500 has 55.00; top tier 2000 has 40.00 and qty_to null
A2	Same item	slider_max 4000; min_qty 100
A3	Custom 3.5×3.5×2, qty 300 (test C1)	Tier 250 unit_price 88.50; tier 500 68.50
A4	Sticker sheets, Paper	slider_max equals max_qty; top tier qty_to equals max_qty
A5	Outdoor, Flex (Star)	Every overpay_from is null; suggest_more false
A6	Any request	All pre-existing fields identical to the baseline snapshot

Slider (M3)

ID	Action	Expected
S1	Open Rigid 3×3×2 Top-Bottom	Markers for every breakpoint, each labelled with quantity and unit price (100 · ₹105, 250 · ₹75, 500 · ₹55 ... 2,000 · ₹40)
S2	Drag to 350	Chip: "350 boxes · ₹75 each" and "Add 150 more → ₹55 each"; 500 marker highlighted; receipt subtotal ₹26,250.00 (T1)
S3	Drag to 366	Not in overpay zone; no better-option banner
S4	Drag to 367 and to 450	Inside amber zone; 500 marker pulses; banner "500 boxes cost ₹6,250 less than 450" at 450 (T5)
S5	Click "Use 500"	Quantity 500, receipt ₹27,500.00 (T2); thumb on the 500 marker
S6	Release thumb at 493	Snaps to 500
S7	Type 60	Thumb in below-minimum segment; billed as 100 at ₹105 with MOQ_APPLIED (T3)
S8	Type 5,000	Thumb pinned at end, "beyond scale" label; tier 2000 at ₹40; no "Add n more" line (T4)
S9	Sticker sheets Paper, type 1,200	Slider ends at max_qty; manual-quote state (T7)
S10	Keyboard: focus slider at 350, Page Up	Quantity 500; Page Down returns to 250; Home to minimum; screen reader reads the value text
S11	Drag steadily from 100 to 2,000 for 3 s	Chip updates continuously; network panel shows no more than one calculate request per 250 ms pause
S12	Outdoor Flex (Star), 10 × 12 ft × 2	Read-only bar at 240 sq ft; no overpay zone or banner (T12)
S13	Custom 3.5×3.5×2 at 300	Markers show custom prices; chip "Add 200 more → ₹68.50 each" (C1)
S14	375 px wide screen	No overlapping labels; thumb target at least 44 px; no pulse with reduced motion on

Navigation (M1)

ID	Action	Expected
N1	Calculator (configured at 450) Cart browser Back	Calculator with the same product, options and 450
N2	Then browser Forward	Cart, same lines
N3	Refresh on /cart and on /checkout	Same screen and data; nothing lost
N4	Paste a calculator URL into a new tab	Same configuration and quote
N5	Move the slider 20 times, then Back once	Leaves the calculator to the previous screen (or site), not to an earlier quantity
N6	Print Paid on checkout, then Back and Forward	No second download, no new invoice number, cart unchanged
N7	Open /does-not-exist	Not-found view with a link to the calculator
N8	URL with an item id that doesn't exist	Calculator default state plus a notice; no crash
N9	Use every in-app back / continue button	Same result as the browser Back button
N10	Production build via <code>npm run preview</code> : refresh on each route	Every route loads; API calls still work

Review pass (M4) and regression (M5)

ID	Check	Expected
R1	Full test suite	Every test in the baseline still passes; new tests added for each fixed bug
R2	Pricing golden tests (T1–T15, C1–C6, L1–L6)	Identical results
R3	Invoice golden rendering tests; Print Unpaid and Print Paid	Identical output
R4	GST selector: all four options	Same tax lines and totals as before
R5	Typecheck and lint (tools already configured)	Zero errors; no more warnings than baseline
R6	Bundle size and API p95	Within Section 8 limits; both reported against baseline
R7	Review report	<code>docs/code-review-report.md</code> complete per FR-C8

12. Open decisions and risks

None of these block the build; each has a default Claude Code should use until Sukrit changes it.

ID	Decision	Default
D1	Send the selected item's tier prices to the browser	Yes. Internal tool; prices already in the shared catalogue; formula stays server-side
D2	Slider end when a product has no <code>max_qty</code>	Top tier × 2 (config <code>slider_max_multiplier</code>)
D3	Snap to breakpoints on release	On, within 1.5% of track width
D4	Which overlays get their own URL	Full-screen overlays only

ID	Decision	Default
D5	Warn before leaving the site with items in the cart	Off (browser prompts annoy daily users)
D6	Keep calculator configuration in the URL	On, so quotes can be re-opened from a link

Risk	Mitigation
Vercel rewrite hides the API or assets	Exclude /api and assets in the rule; N10 locally; re-check on a preview deploy after approval
Cart or checkout data lost on route change	State lives above the router (FR-B3); N1-N3
History filled with slider steps	URL updates use replace (FR-B2); N5
Marker labels collide on items with many tiers	Price-only labels and tap tooltips on narrow widths (FR-A12); S14
Review pass changes behaviour	Frozen outputs (FR-C2), baseline comparison, golden tests (R1-R4), ask-first list (FR-C6)
Chip and receipt disagree	Both derive from server data; server wins (FR-A11)

Appendix: kickoff prompt for Claude Code

Save this PDF in the repo as docs/BRD-tier-slider-back-nav.pdf, then paste:

```
Read docs/BRD-tier-slider-back-nav.pdf in full before doing anything.
Follow the hard rules in Section 9 exactly: work on branch feature/tier-slider-back-nav,
local commits per milestone are fine, but never push, merge or deploy.
Build milestones M0 to M5 from Section 10 in order. Run the full test suite at the end of
each milestone and do not start the next until it passes. Post a short summary after each
milestone. Stop and ask only if something conflicts with the BRD.
After M5, give me the localhost URLs, the UAT checklist and the review report, then wait
for my approval.
```